

算法设计与分析

Design & Analysis of Algorithms

第3章 动态规划

学习要点

- 理解动态规划算法的概念
- 掌握动态规划算法的基本要素
 - (1) 最优子结构性质 (2) 重叠子问题性质
- 掌握设计动态规划算法的步骤
- 通过应用范例学习动态规划算法设计策略
 - (1) 基础题目
 - (2) 矩阵连乘问题
 - (3) 背包问题
 - (4) 子序列问题

3.4 背包问题 问题描述

- 有 n 个物品，每个物品有重量 $weight$ 和价值 $profit$ 两个属性，还有一个承重为 c 的背包。现在让你用这个背包装物品，使所装物品的价值最大化。

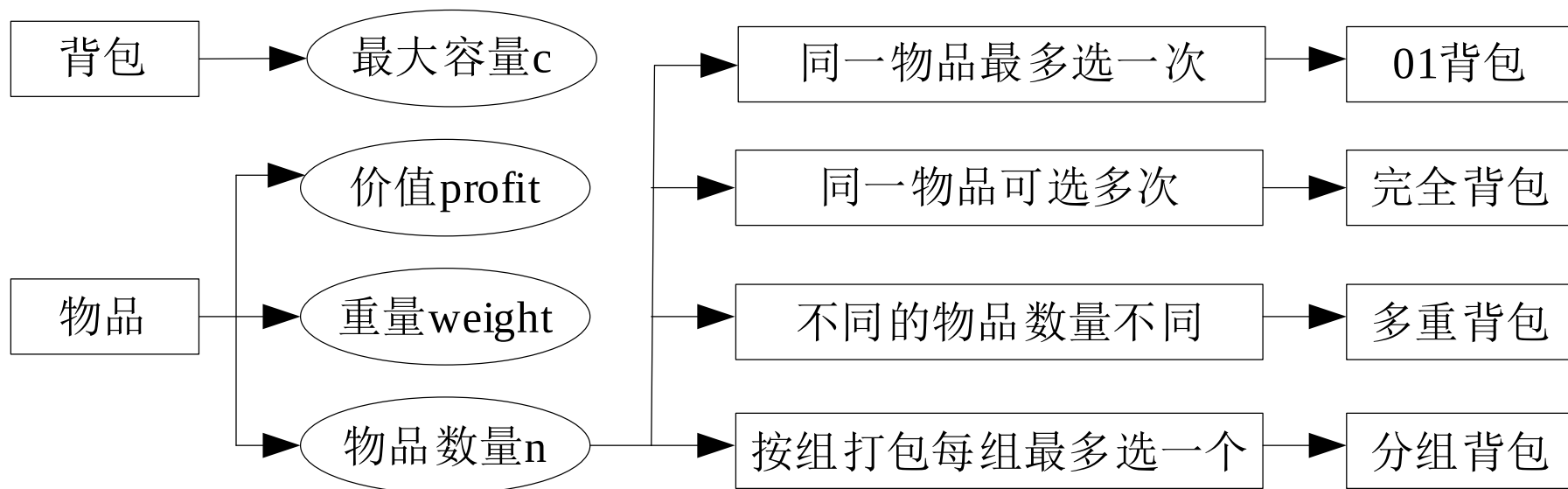
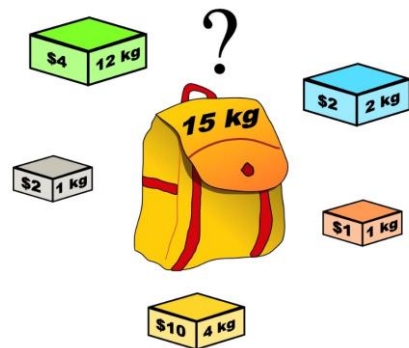
例子：背包容量：5

- 尝试总重量不超过5：

- 苹果+橙（总重量5） $\Rightarrow$ 9价值
- 苹果+香蕉（总重量3） $\Rightarrow$ 7价值
- 橙+香蕉（总重量4） $\Rightarrow$ 8价值
- 香蕉+瓜（总重量5） $\Rightarrow$ 10价值

香蕉+甜瓜是最好的组合，因为带来最大的价值，并且总重量不超过背包的承重量。

	重量	价值
苹果	2	4
橙子	3	5
香蕉	1	3
甜瓜	4	7



3.4 背包问题 01背包蛮力算法

for 每个物品 i:

如果总重量不超过C，则创建一个包含物品i的集合，然后在剩余的物品和承重量上递归调用该方法；

否则创建一个不包含物品i的集合，然后在剩余的物品和承重量上递归调用该方法；

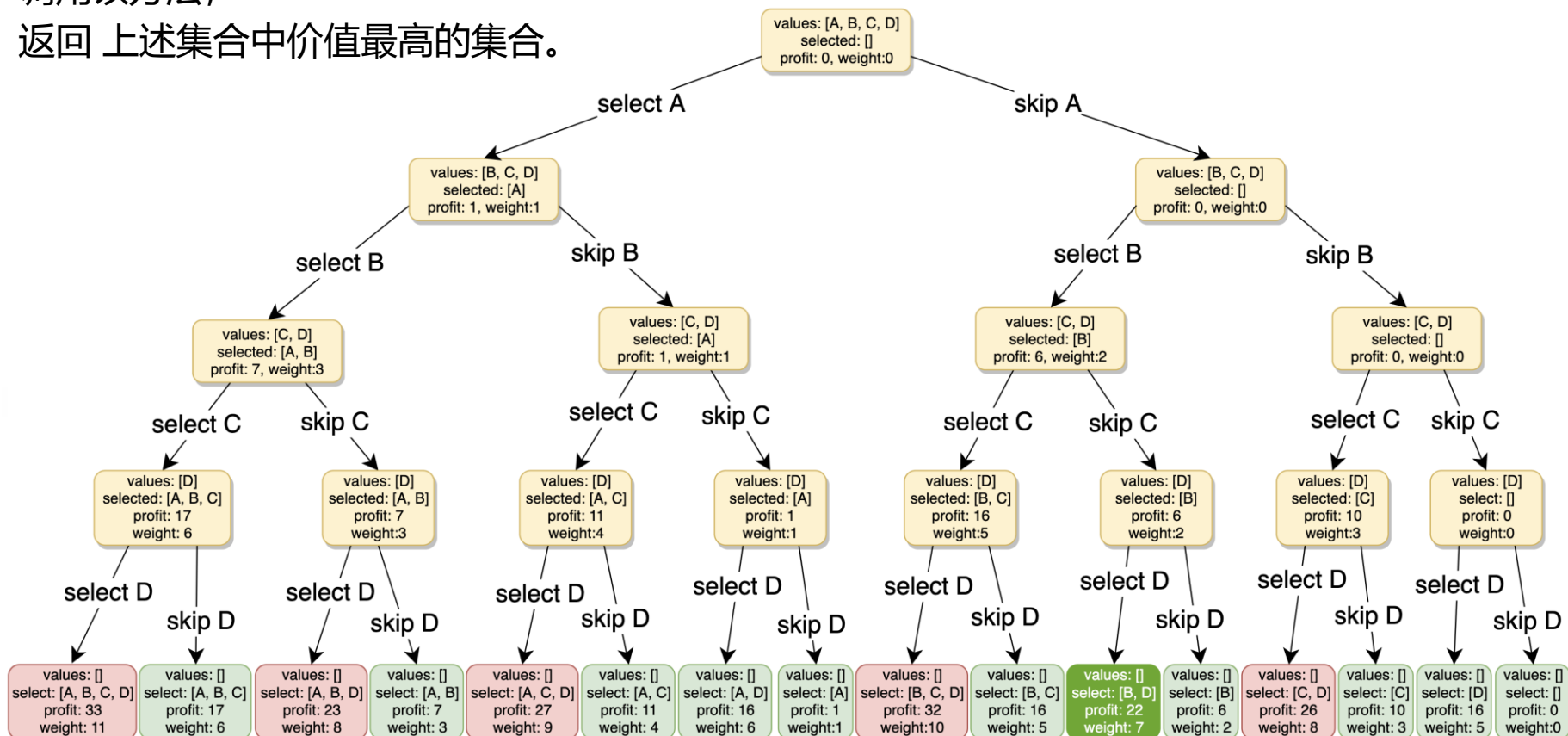
返回 上述集合中价值最高的集合。

所有红色的集合都是超重的集合。其余的集合都是满足背包承重要求的（本例中是7）。最佳的选项是[B,D]，总价值是22，总重量恰好为7。

- 时间复杂度是 $O(2^n)$
- 空间复杂度是 $O(n)$ 栈最深的递归调用长度

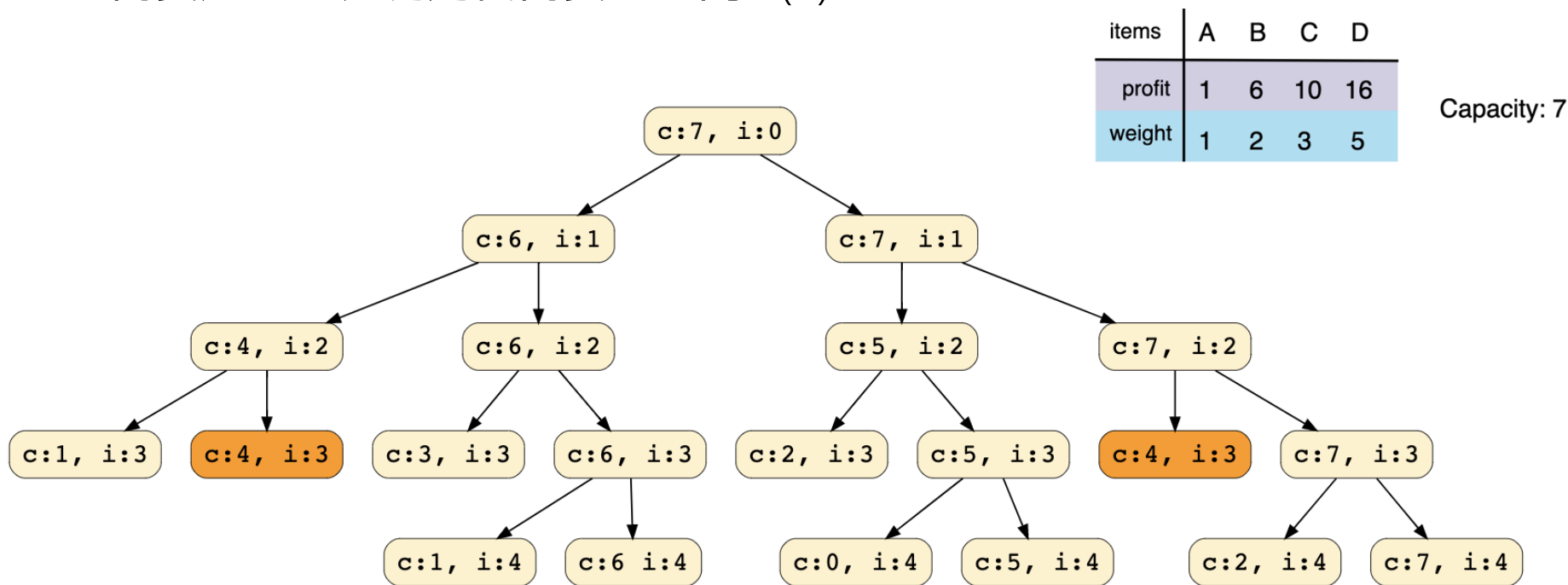
items	A	B	C	D
profit	1	6	10	16
weight	1	2	3	5

Capacity: 7



3.4 背包问题 01背包自顶向下加备忘录(递归)

- 递归调用方法中，profit和weight两个数组是保持不变的，变化的是当前剩余承重量和当前元素位置，分别用c和i来表示。
- 重叠的子问题，“c:4, i=3”被计算了两次. 用一个二维数组来存储子问题解。
- 时间复杂度 $O(n*c)$ 最多需要计算的子问题的次数不超过 $n*c$ ，n是物品数量，c是背包容量。
- 空间复杂度 $O(n*c)$ 缓存子问题的结果需要的二维数组的空间 $O(n*c)$ 。此外，还需要加上递归调用栈需要的空间 $O(n)$ 。



3.4 背包问题 01背包 动态规划

- 确定dp数组以及下标的含义

二维数组 $dp[i][j]$, 表示从下标为 $[0-i]$ 的物品里任意取, 放进容量为 j 的背包, 价值总和最大。

			capacity -->							
profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0							
6	2	1	0							
10	3	2	0							
16	5	3	0							

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包 动态规划

- 确定递推公式 状态转移方程

-不放物品*i*: 由 $dp[i-1][j]$ 推出, 即背包容量为*j*, 里面不放物品*i*的最大价值, 此时 $dp[i][j]$ 就是 $dp[i-1][j]$ 。(其实就是当物品*i*的重量大于背包*j*的重量时, 物品*i*无法放进背包中, 所以背包内的价值依然和前面相同。)

-放物品*i*: 由 $dp[i-1][j - \text{weight}[i]]$ 推出, $dp[i-1][j - \text{weight}[i]]$ 为背包容量为 $j - \text{weight}[i]$ (空出空间好放*i*)时不放物品*i*的最大价值, 则 $dp[i-1][j - \text{weight}[i]] + \text{profit}[i]$ (物品*i*的价值)是放物品*i*得到的最大价值。

递归公式: $dp[i][j] = \max(dp[i-1][j], dp[i-1][j - \text{weight}[i]] + \text{profit}[i]);$

			capacity -->							
profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0	1	6					
10	3	2	0							
16	5	3	0							

$$dp[1][2] = \max(dp[0][2], \text{profit}[1] + dp[0][0])$$

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包 动态规划

- 确定递推公式 状态转移方程

-不放物品*i*: 由 $dp[i-1][j]$ 推出, 即背包容量为*j*, 里面不放物品*i*的最大价值, 此时 $dp[i][j]$ 就是 $dp[i-1][j]$ 。(其实就是当物品*i*的重量大于背包*j*的重量时, 物品*i*无法放进背包中, 所以被背包内的价值依然和前面相同。)

-放物品*i*: 由 $dp[i-1][j-\text{weight}[i]]$ 推出, $dp[i-1][j-\text{weight}[i]]$ 为背包容量为 $j-\text{weight}[i]$ (空出空间好放*i*)时不放物品*i*的最大价值, 则 $dp[i-1][j-\text{weight}[i]] + \text{profit}[i]$ (物品*i*的价值)是放物品*i*得到的最大价值。

递归公式: $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-\text{weight}[i]] + \text{profit}[i]);$

		capacity -->							
index		0	1	2	3	4	5	6	7
dp[i-1][j-weight[i]]+profit[i]	0	0	1	1	1	1	1	1	1
	2	0							
	3	0							

Diagram illustrating the DP table structure and state transitions:

- The table has rows for index (0, 2, 3) and columns for capacity (0 to 7).
- Row 0 (index 0) contains values: 0, 1, 1, 1, 1, 1, 1, 1.
- Row 2 (index 2) contains values: 0, followed by empty cells.
- Row 3 (index 3) contains values: 0, followed by empty cells.
- A red box highlights the cell at index 2, capacity 2, labeled $dp[i-1][j-\text{weight}[i]] + \text{profit}[i]$.
- A blue box highlights the cell at index 2, capacity 5, labeled $dp[i-1][j]$.
- A red arrow points from the red box to the blue box, indicating the transition from the state where item *i* is not taken to the state where item *i* is taken.
- A blue box highlights the cell at index 3, capacity 5, labeled $dp[i][j]$.

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包 动态规划

3. dp数组如何初始化

➤ $dp[i][0]$, 即背包容量 j 为0, 无论是选取哪些物品, 背包价值总和一定为0;

➤ $dp[0][j]$, 即 i 为0, 存放编号0的物品, 各个容量的背包所能存放的最大价值;

当 $j < \text{weight}[0]$, $dp[0][j]$ 是0,

当 $j \geq \text{weight}[0]$, $dp[0][j]$ 是 $\text{profit}[0]$ 。

```
vector<vector<int>>dp(weight.size(), vector<int>(c + 1, 0));
```

```
for (int j = weight[0]; j <= c; j++) {dp[0][j] = profit[0]; }
```

			capacity -->							
profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0							
10	3	2	0							
16	5	3	0							

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包 动态规划

- 确定遍历顺序

```
for(int i = 1; i < weight.size(); i++) { // 遍历物品
    for(int j = 0; j <= c; j++) { // 遍历背包容量
        if (j < weight[i]) dp[i][j] = dp[i - 1][j];
        else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + profit[i]);
    }
}
```

```
for(int j = 0; j <= c; j++) { // 遍历背包容量
    for(int i = 1; i < weight.size(); i++) { // 遍历物品
        if (j < weight[i]) dp[i][j] = dp[i - 1][j];
        else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - weight[i]] + profit[i]);
    }
}
```

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

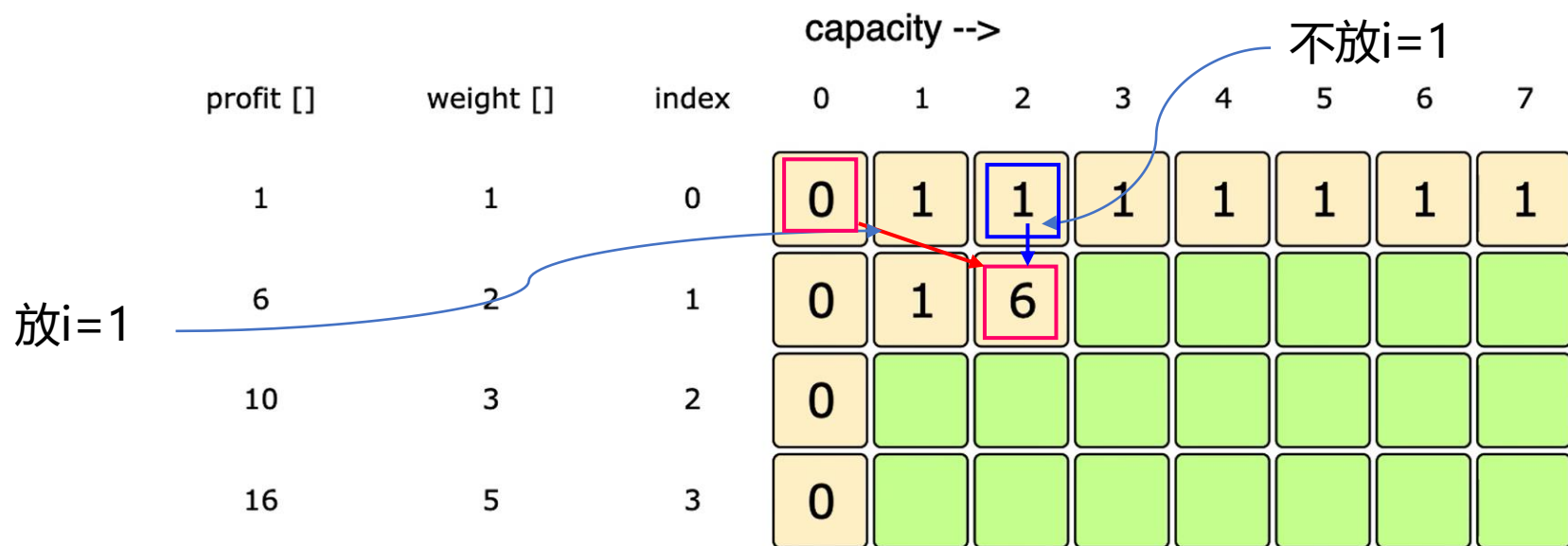
capacity -->

profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0							
6	2	1	0							
10	3	2	0							
16	5	3	0							

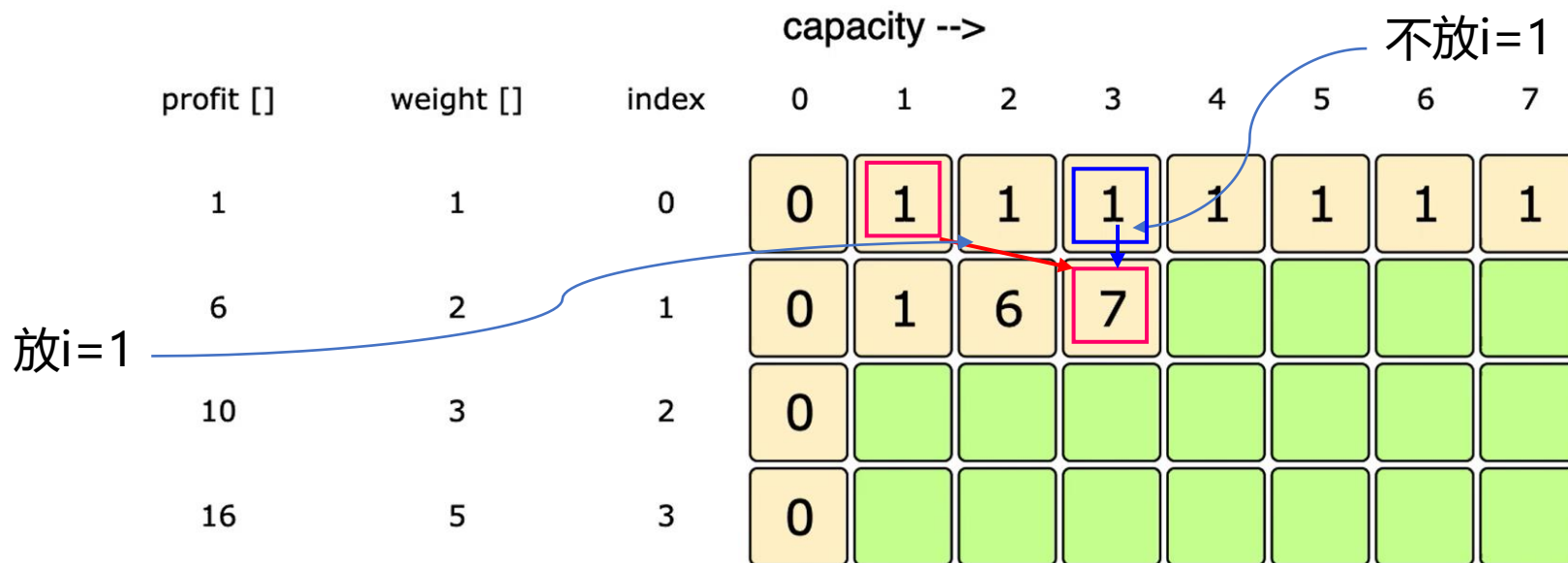
■ 初始化

capacity -->

profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0							
10	3	2	0							
16	5	3	0							



- $\max(dp[0][2], profit[1] + dp[0][0]), \text{ Capacity}=2, \text{ Index}=1$



- $\max(dp[0][3], profit[1] + dp[0][1]), \text{ Capacity}=3, \text{ Index}=1$

capacity -->

profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0	1	6	7	7	7	7	7
10	3	2	0	1	6	10				
16	5	3	0							

放i=2

不放i=2

- max(dp[1][3], profit[2]+dp[1][0]), Capacity=3, Index=2

capacity -->

profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0	1	6	7	7	7	7	7
10	3	2	0	1	6	10	11			
16	5	3	0							

- max(???, ??? + ???), Capacity=4, Index=2

				capacity -->							
profit []	weight []	index		0	1	2	3	4	5	6	7
1	1	0		0	1	1	1	1	1	1	1
6	2	1		0	1	6	7	7	7	7	7
10	3	2		0	1	6	10	11	16		
16	5	3		0							

- Capacity=5, Index=2, $\max(dp[1][5], profit[2]+dp[1][2])$

				capacity -->							
profit []	weight []	index		0	1	2	3	4	5	6	7
1	1	0		0	1	1	1	1	1	1	1
6	2	1		0	1	6	7	7	7	7	7
10	3	2		0	1	6	10	11	16	17	17
16	5	3		0	1	6	10	11	16	17	22

- Capacity=7, Index=3, $\max(dp[2][7], profit[3]+dp[2][2])$

最优解

3.4 背包问题 01背包 动态规划

- 最优解追踪

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j - \text{weight}[i]] + \text{profit}[i]);$$

- 记录决策过程

$\text{rec}[i][j]=1$, 选择商品

0, 不选商品

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

		capacity -->							
index		0	1	2	3	4	5	6	7
0		0	1	1	1	1	1	1	1
1		0							
2		0							
3		0							

Diagram illustrating the DP table and decision recording:

- The table shows the maximum profit for each item (index 0 to 3) and capacity (0 to 7).
- Cells with index 0 are yellow and contain values 0 or 1.
- Other cells are green.
- Red boxes highlight specific cells: $\text{rec}[i][j]=1$ at (2,2) and $\text{rec}[i][j]=0$ at (1,5).
- A red arrow points from the cell at (2,2) to the cell at (2,5), which is labeled $\text{dp}[i][j]$.

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包 动态规划

- 最优解追踪

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j - \text{weight}[i]] + \text{profit}[i]);$$

- 回溯解决方案 倒序判断是否选择商品

$\text{rec}[i][j]=1$, 选择商品

0, 不选商品

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

capacity -->								
index	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0							
2	0							
3	0							

Diagram illustrating the DP table for the 01 Knapsack problem. The table has 4 rows (index 0 to 3) and 9 columns (capacity 0 to 8). The first column (capacity 0) is labeled 'index' and contains values 0, 1, 2, 3. The first row (index 0) is labeled 'capacity -->' and contains values 0, 1, 2, 3, 4, 5, 6, 7. The cells are colored: yellow for index 0, green for index 1, 2, 3. A red box highlights the cell at index 1, capacity 2, labeled $\text{rec}[i][j]=1$. A blue arrow points to the cell at index 1, capacity 5, labeled $\text{rec}[i][j]=0$. A red arrow points from the red box to the cell at index 2, capacity 4, labeled $\text{dp}[i][j]$.

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

- 算法实例 $dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - \text{weight}[i]] + \text{profit}[i]);$

capacity -->

index	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	1	6					
2	0							
3	0							

包含商品 1

rec	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	0	1					
2	0							
3	0							

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

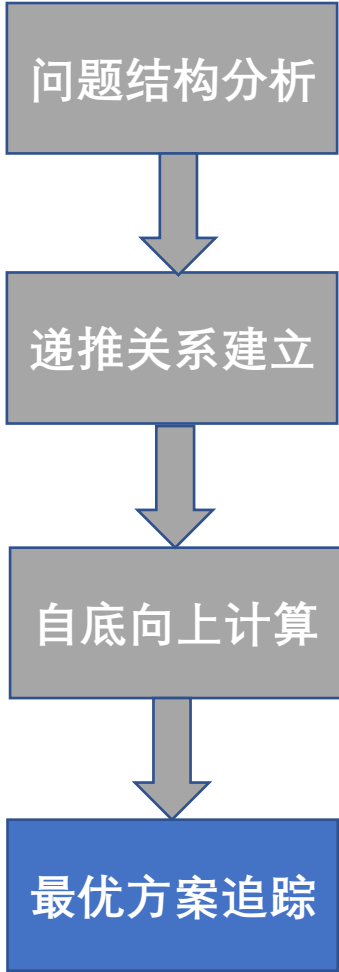
- 算法实例 $dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - \text{weight}[i]] + \text{profit}[i]);$

capacity -->

index	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	1	6	7	7	7	7	7
2	0	1	6	10	11	16	17	17
3	0	1	6	10	11	16	17	22

选取的商品：
3

rec	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	1
2	0	0	0	1	1	1	1	1
3	0	0	0	0	0	0	0	1



最优解包含商品 3

3.4 背包问题 01背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

- 算法实例 $dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - \text{weight}[i]] + \text{profit}[i]);$

capacity -->

index	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	1	6	7	7	7	7	7
2	0	1	6	10	11	16	17	17
3	0	1	6	10	11	16	17	22

选取的商品:
3

rec	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	1
2	0	0	0	1	1	1	1	1
3	0	0	0	0	0	0	0	1

最优解不包
含商品 2

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

- 算法实例 $dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - \text{weight}[i]] + \text{profit}[i]);$

capacity -->

index	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	1	6	7	7	7	7	7
2	0	1	6	10	11	16	17	17
3	0	1	6	10	11	16	17	22

选取的商品：
3, 1

rec	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	1
2	0	0	0	1	1	1	1	1
3	0	0	0	0	0	0	0	1

最优解包含
商品 1

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

- 算法实例 $dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - \text{weight}[i]] + \text{profit}[i]);$

capacity -->

index	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	1	6	7	7	7	7	7
2	0	1	6	10	11	16	17	17
3	0	1	6	10	11	16	17	22

选取的商品：
3, 1

rec	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	1
2	0	0	0	1	1	1	1	1
3	0	0	0	0	0	0	0	1

最优解不包
含商品 1

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.4 背包问题 01背包

滚动数组 由二维数组简化到一维数组

- 确定dp数组以及下标的含义
dp[j]: 容量为j的背包中物品最大价值。
- 确定递推公式, 状态转移方程
dp[j]: dp[j] 相当于二维dp数组中的dp[i-1][j], 不放物品i.
dp[j - weight[i]] + profit[i], 放物品i.
dp[j]=max(dp[j], dp[j-weight[i]]+profit[i]);
- dp数组初始化
dp[j]:都初始化为0.

- 确定遍历顺序
遍历顺序从大到小, 否则物品会重复加入

```
for(int i = 0; i < weight.size(); i++) { // 遍历物品
    for(int j = c; j >= weight[i]; j--)
    { // 遍历背包容量
        dp[j] = max(dp[j], dp[j - weight[i]] + profit[i]);
    }
}
```

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

		capacity -->							
index		0	1	2	3	4	5	6	7
0		0	1	1	1	1	1	1	1
1		0	1	6	7	7	7	7	7
2		0	1	6	10	11	16	17	17
3		0	1	6	10	11	16	17	22

实验3-3：01背包问题

- 问题描述：有n个物品，每个物品有重量weight和价值profit两个属性，还有一个承重为c的背包。现在让你用这个背包装物品，使所装物品的价值最大化。

- 示例 1：

输入：

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

输出：

- 要求：

- 1 填空完成；
- 2 打印dp数组；
- 3 打印追踪数组rec[]。

3.4 背包问题 完全背包

- 问题描述：有n件物品和一个最多能背重量为c的背包。第i件物品的重量是weight[i]，得到的价值是profit[i]。每件物品都有无限个（也就是可以放入背包多次），求解将哪些物品装入背包里物品价值总和最大。
- 01背包 遍历次序

```
for(int i = 0; i < weight.size(); i++) { // 遍历物品
    for(int j = c; j >= weight[i]; j--)
    { // 遍历背包容量
        dp[j] = max(dp[j], dp[j - weight[i]] + profit[i]);
    }
}
```


3.4 背包问题 完全背包

滚动数组 由二组数组简化到一组数组

- 确定遍历顺序

- 为了保证每个物品仅被添加一次，01背包内循环从大到小遍历,否则物品会重复加入。

- 完全背包的物品是可以添加多次的，所以要从小到大去遍历。

- 两个for循环嵌套，不同遍历顺序结果相同。

```
// 先遍历物品，再遍历背包
```

```
for (int i = 0; i < weight.size(); i++) { // 遍历物品
    for (int j = 1; j <= c; j++) { // 遍历背包容量
        if (j < weight[i])    dp[j] = dp[j];
        else    dp[j] = max(dp[j], dp[j - weight[i]] + profit[i]);
    }
}
```

```
for(int i = 0; i < weight.size(); i++) { // 遍历物品
    for(int j = weight[i]; j <= c ; j++) { // 遍历背包容量
        dp[j] = max(dp[j], dp[j - weight[i]] + profit[i]);
    }
}
```

3.4 背包问题 完全背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

- 实例

// 先遍历物品，再遍历背包

```
for (int i = 0; i < weight.size(); i++) { // 遍历物品
    for (int j = 1; j <= c; j++) { // 遍历背包容量
        if (j < weight[i])    dp[j] = dp[j];
        else    dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
}
```

	j=0	1	2	3	4	5	6	7
i=0	0	1	2	3	4	5	6	7
1								
2								
3								

3.4 背包问题 完全背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

- 实例

// 先遍历物品，再遍历背包

```
for (int i = 0; i < weight.size(); i++) { // 遍历物品
```

```
    for (int j = 1; j <= c; j++) { // 遍历背包容量
```

```
        if (j < weight[i])    dp[j] = dp[j];
```

```
        else dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
```

```
    }
```

```
}
```

	j=0	1	2	3	4	5	6	7
i=0	0	1	2	3	4	5	6	7
1	0	1	6					
2								
3								

Diagram illustrating the DP table for the complete knapsack problem. The table has rows for items (i) and columns for capacity (j). The value in the cell (i=1, j=2) is 6, which is highlighted with a red box. An arrow points from the label $dp[2]$ to this cell. Another arrow points from the value 1 in the cell (i=1, j=1) to the value 6 in the cell (i=1, j=2).

$i=1 \quad dp[2] = \max(dp[2], dp[2 - \text{weight}[1]] + \text{value}[1]) = dp[0] + 6$

3.4 背包问题 完全背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

- 实例

// 先遍历物品，再遍历背包

```
for (int i = 0; i < weight.size(); i++) { // 遍历物品
```

```
    for (int j = 1; j <= c; j++) { // 遍历背包容量
```

```
        if (j < weight[i])    dp[j] = dp[j];
```

```
        else    dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
```

```
    }
```

```
}
```

dp[4]

	j=0	1	2	3	4	5	6	7
i=0	0	1	2	3	4	5	6	7
1	0	1	6	7	12			
2								
3								

i=1 $dp[4] = \max(dp[4], dp[4 - \text{weight}[1]] + \text{value}[1]) = dp[2] + 6 = 12$

3.4 背包问题 完全背包

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

- 实例

// 先遍历物品，再遍历背包

```
for (int i = 0; i < weight.size(); i++) { // 遍历物品
    for (int j = 1; j <= c; j++) { // 遍历背包容量
        if (j < weight[i])    dp[j] = dp[j];
        else    dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
}
```

	j=0	1	2	3	4	5	6	7
i=0	0	1	2	3	4	5	6	7
1	0	1	6	7	12	13	18	19
2	0	1	6	10	12	?		
3								22

实验3-4：完全背包

- 问题描述：有n件物品和一个最多能背重量为c的背包。第i件物品的重量是weight[i]，得到的价值是profit[i]。每件物品都有无限个（也就是可以放入背包多次），求解将哪些物品装入背包里物品价值总和最大。

- 示例 1：

输入：

items	A: 0	B: 1	C: 2	D: 3
profit	1	6	10	16
weight	1	2	3	5

输出：

- 要求：

- 1 填空完成程序；
- 2 打印dp数组；
- 3 打印追踪数组rec[]。